🚀 C++ GDB 面试速通背诵版 (两年经验专属)

核心人设（面试基调）： “我平时用 GDB 主要做**线上排障**和**Core Dump分析**。遇到崩溃先看 `bt` 切栈帧，看局部变量；多线程卡死就看所有线程栈状态。复杂的内存越界我还会配合日志和 ASan 一起查。”（这段话直接背，开场甩出来）

💥 场景一：面试必问的 3 大实战套路（直接背诵）

1. “程序段错误 (Segfault) / Core Dump 怎么查？”

- **第一步：加载环境** -> `gdb ./a.out core`（前提是 `ulimit -c unlimited` 开了）。
- **第二步：看全貌** -> 输入 `bt` (backtrace) 看完整的函数调用栈。
- **第三步：找案发现场** -> 用 `frame 0` 或 `up/down` 切换到崩溃的那层栈帧。
- **第四步：查变量** -> 用 `info locals`（看局部变量）、`info args`（看参数），或者直接 `p 变量名` 看指针是不是空指针或野指针。
- **话术总结：** “先 bt 看栈，切到崩溃帧查局部变量，顺着调用链找是谁传了坏数据。”

2. “线上 CPU 100% / 死锁 / 死循环 怎么查？”

- **第一步：附加进程** -> `top` 查到 PID，然后 `gdb -p <pid>` 挂载上去。
- **第二步：看所有线程** -> 输入 `info threads` 看线程列表。
- **第三步：一键看全部** -> 输入 `thread apply all bt` 打印所有线程的调用栈。
- **分析话术：** “如果是死锁，通常会看到几个线程都卡在 `std::mutex::lock` 或者等待条件变量上；如果是死循环，会看到某个线程一直卡在同一个业务函数的代码行（配合 `c` 和 `Ctrl+C` 多看几次）。”

3. “有个变量的值莫名其妙被改坏了，怎么查？”

- **杀手锏：打硬件观察点 (Watchpoint)。**
- **话术：** “我会先停在变量初始化的地方，然后用 `watch x` 给它打个观察点。接着输入 `c` (continue) 让程序跑，一旦有代码去修改 `x`，GDB 就会自动停住，这时候打个 `bt` 就能抓到是谁（哪个函数）越界写了它。”

🛠️ 场景二：面试常问的 3 个细节点（避坑必背）

1. **`next` (n) 和 `step` (s) 的区别是什么？**
   - **秒答：** `next` 是单步执行，**不进**入函数内部；`step` 也是单步，但如果遇到函数调用，会**进入**函数内部。
2. **为什么 GDB 调试时，提示变量 `optimized out` (查不到变量)？**
   - **秒答：** 因为编译的时候开启了 `-O2` 或 `-O3` 优化，变量被放进寄存器或者被优化掉了。
   - **补充：** “所以为了排障方便，测试环境编译通常带 `-g`（加符号表）和 `-O0`（关优化）。”

3. **如何下条件断点？**
    - **秒答：** `b test.cpp:30 if x == 5` 。遍历大数组或者在循环里排错时特别好用。

⌨️ 极简命令清单 (考前看一眼，形成肌肉记忆)

1. 启动三板斧

- `gdb ./a.out` (常规调)
- `gdb ./a.out core` (查崩溃)
- `gdb -p pid` (查线上挂载)

2. 断点与运行 (BRN S/C)

- `b main` / `b file.cpp:10` (打断点)
- `r` (run，跑起来)
- `n` (next，单步不进)
- `s` (step，单步要进)
- `c` (continue，继续跑直到下一个断点)

3. 查变量与栈 (BFP I)

- `bt` (看调用栈，极其重要)
- `frame 0` / `f 0` (切到第0帧)
- `p x` (打印变量)
- `p *ptr` (打印指针指向的内容)
- `info locals` (看当前所有局部变量)

4. 多线程 (必杀技)

- `info threads`
- `thread apply all bt` (遇到卡死直接敲这个)